Análise Comparativa entre Multilayer Perceptron e Naive Bayes Gaussiano em Cenários de Classificação Binária e Multiclasse

Luís Emílio Gonçalves Mendes
Centro de Matemática, Computação e Cognição (CMCC)
Universidade Federal do ABC (UFABC)
Santo André, Brasil
luis.emilio@aluno.ufabc.edu.br

Nicholas Fernandez Garcia
Centro de Matemática, Computação e Cognição (CMCC)
Universidade Federal do ABC (UFABC)
Santo André, Brasil
fernandez.garcia@aluno.ufabc.edu.br

***Resumo*—Este artigo apresenta uma análise comparativa de desempenho entre dois algoritmos de aprendizado supervisionado: o modelo Multilayer Perceptron (MLP) treinado com algoritmo *backpropagation* via descida de gradiente estocástico, e o classificador probabilístico Naive Bayes Gaussiano. Ambos os algoritmos foram desenvolvidos do zero em Python e submetidos a duas bases de dados com características distintas: o conjunto *Titanic*, que representa um cenário de classificação binária, e o conjunto *Pokémon*, caracterizado por um espaço multiclasse com 18 classes. A capacidade de generalização dos modelos foi avaliada sob divisão estratificada por meio de acurácia global, precisão, *recall*, *F1-Score*, matrizes de confusão e convergência de perda. Os resultados demonstram equivalência preditiva no cenário binário (79,89% na MLP e 78,77% no Naive Bayes), enquanto no multiclasse a flexibilidade não linear da MLP superou a hipótese de independência condicional do Naive Bayes (35,17% contra 17,24%), indicando que, enquanto a MLP se sobressai em tarefas complexas mediante um maior consumo de recursos computacionais, o Naive Bayes consolida-se como uma opção leve e de baixo custo para cenários menos complexos.**

Index Terms*—Multilayer Perceptron, *Backpropagation*, Naive Bayes, *Pokémon*, *Titanic

I. Introdução

Modelos de aprendizado de máquina supervisionado desempenham papel fundamental na resolução de problemas, possuindo a capacidade de modelar cenários complexos da natureza através de funções. Nesse contexto, dentro desses modelos estão os classificadores, que têm como finalidade associar vetores de entrada por meio de atributos selecionados e relacioná-los a categorias de saída pré-definidas.

O objetivo deste trabalho é analisar e comparar o desempenho de dois classificadores (Multilayer Perceptron (MLP) e Naive Bayes Gaussiano) em duas bases de dados distintas: a base *Titanic* [1], que possui um problema de classificação binário de sobrevivência, e a base *Pokémon* [2], um problema multiclasse (18 classes), voltado à classificação do tipo primário do *Pokémon*. A avaliação dos modelos é realizada por meio de métricas de acurácia, precisão, *recall*, *F1-Score*, matrizes de confusão e curva de aprendizado de erro.

O restante deste artigo está organizado da seguinte forma: a Seção II aborda a fundamentação teórica dos modelos avaliados; a Seção III descreve todo o processo de desenvolvimento, desde a coleta de dados até a avaliação dos modelos; a Seção IV apresenta e discute os resultados obtidos em cada modelo e o que eles significam; e a Seção V recapitula o objetivo do trabalho, retoma a metodologia e os resultados obtidos e faz uma análise final sobre eles.

II. Referencial Teórico

Este trabalho consiste em um estudo experimental comparativo sobre o desempenho de dois algoritmos de classificação em bases de dados distintas. Entre os diversos modelos disponíveis, como *k-Nearest Neighbours*, árvores de decisão e máquinas de vetores de suporte (SVM), foram escolhidos a rede neural artificial MLP com *backpropagation* e o classificador Naive Bayes Gaussiano por serem abordagens diferentes, uma baseada em otimização de parâmetros iterativa e o outro em estimativas probabilísticas.

Uma das principais vantagens da MLP está no fato de ser muito flexível, podendo modelar relações não lineares. Em contrapartida, possui alto custo computacional necessário para o treinamento [3], além da complexidade para definir os hiperparâmetros. Já o Naive Bayes baseia-se no cálculo de probabilidades assumindo independência condicional dada a classe, o que raramente é verdadeiro em dados reais. Ainda assim, costuma ter bom desempenho e consome menos recursos dada a simplicidade do cálculo e do treinamento. Além disso, a variante Gaussiana assume que cada atributo segue uma distribuição normal em cada classe, o que não se verifica para atributos assimétricos ou binários, e depende de estimativas de média e desvio padrão que se tornam instáveis em classes com poucas amostras.

As fórmulas matemáticas a seguir fundamentam-se na literatura clássica de aprendizado supervisionado [3].

A. Multilayer Perceptron com Backpropagation

A MLP é uma rede neural composta por uma camada de entrada, uma ou mais camadas ocultas e uma camada de saída. A camada de entrada é responsável por receber os dados iniciais, já as camadas ocultas e de saída são compostas por

neurônios. Cada neurônio é responsável por multiplicar os valores de entrada (x_i) por diferentes pesos (w_i) e somá-los a um viés (b), como mostra a Equação 1.

$$s = \sum_{i=1}^{n} x_i w_i + b \tag{1}$$

O resultado do somatório (s) é aplicado em uma função de ativação, que neste trabalho foi a sigmoide, dada por:

$$\sigma(s) = \frac{1}{1 + e^{-s}} \tag{2}$$

Esse processo de propagar as informações de entrada até a saída é chamado de *feedforward* e resulta em uma predição $\hat{y}$. Em seguida, o erro da predição é calculado comparando-a com o valor esperado y, por meio da função de erro para uma amostra:

$$E = \frac{1}{2} \left\| y - \hat{y} \right\|^2 \tag{3}$$

Calcula-se então o gradiente, isto é, as derivadas parciais desse erro em relação aos pesos, que pela regra da cadeia são dadas por:

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial s} \cdot \frac{\partial s}{\partial w_i} \tag{4}$$

em que $\partial \hat{y} / \partial s$ envolve a derivada da sigmoide:

$$\sigma'(s) = \sigma(s)\big(1 - \sigma(s)\big) \tag{5}$$

O gradiente é propagado para as camadas anteriores através da regra da cadeia, processo denominado *backpropagation*. Como o gradiente indica a direção de maior crescimento da função do erro, os pesos são atualizados na direção oposta através da função:

$$w_i \leftarrow w_i - \eta \frac{\partial E}{\partial w_i} \tag{6}$$

em que η é a taxa de aprendizado e o sinal negativo indica a direção oposta ao gradiente. Esse procedimento é repetido ao longo de várias épocas até que um critério de parada, definido na Seção III, seja atingido.

B. Classificador Naive Bayes Gaussiano

O Naive Bayes é um classificador baseado no Teorema de Bayes que calcula a probabilidade de uma amostra $x = (x_1, \ldots, x_d)$ pertencer a uma classe C_k por meio de:

$$P(C_k \mid x) = \frac{P(x \mid C_k)\, P(C_k)}{P(x)} \tag{7}$$

Para tornar o cálculo de $P(x \mid C_k)$ viável, assume-se que os atributos são condicionalmente independentes dada a classe, o que permite escrever:

$$P(x \mid C_k) = \prod_{i=1}^{d} P(x_i \mid C_k) \tag{8}$$

O nome *naive* (ingênuo) vem do fato de que essa suposição dificilmente é verdadeira em dados reais. Neste trabalho adotou-se a variante Gaussiana, na qual cada $P(x_i \mid C_k)$ é modelada por uma distribuição normal

$$P(x_i \mid C_k) = \frac{1}{\sigma_{k,i}\sqrt{2\pi}} \exp\left(-\frac{(x_i - \mu_{k,i})^2}{2\sigma_{k,i}^2} \right) \tag{9}$$

em que $\mu_{k,i}$ é a média e $\sigma_{k,i}$ o desvio padrão do atributo i entre as amostras da classe C_k. No treinamento, esses parâmetros e as probabilidades a priori $P(C_k)$ são estimados a partir dos dados. Na classificação, como $P(x)$ é o mesmo para todas as classes, a amostra é atribuída à classe de maior probabilidade a posteriori. Na prática, esse cálculo é feito com logaritmos, para evitar *underflow* numérico causado pelo produtório de muitas probabilidades pequenas:

$$\hat{C} = \arg\max_k \left[\ln P(C_k) + \sum_{i=1}^{d} \ln P(x_i \mid C_k) \right] \tag{10}$$

onde $\hat{C}$ é a classe predita.

III. METODOLOGIA

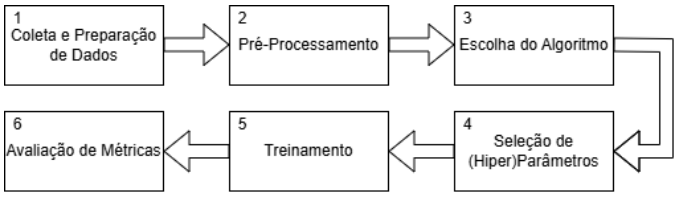


Figura 1: Fluxograma da metodologia experimental adotada.

A ideia geral da metodologia proposta neste projeto é composta por 6 módulos, conforme ilustrado na Figura 1. Todos os modelos foram implementados na linguagem Python (versão 3.8+), utilizando a biblioteca NumPy [4] exclusivamente para operações de álgebra linear e manipulação matricial vetorizada dos algoritmos. O pré-processamento tabular foi conduzido com a biblioteca pandas [5], enquanto o pacote scikit-learn [6] foi empregado estritamente para a partição estratificada das amostras e a geração das matrizes de confusão. A renderização das curvas de aprendizado e matrizes gráficas foi realizada com a biblioteca Matplotlib [7], garantindo que o núcleo decisório de ambos os classificadores fosse desenvolvido de forma autônoma.

A. Coleta, Preparação e Pré-Processamento dos Dados

O Módulo 1 consiste na escolha das duas bases de dados que serão utilizadas, neste trabalho as bases escolhidas foram: o conjunto de dados *Titanic* [1], o qual é voltado à predição de sobrevivência em um cenário binário, e o conjunto *Pokémon* [2], cujo objetivo é a classificação do tipo primário de 721 *Pokémon* distribuídos em 18 classes distintas, por meio de seus atributos físicos e estatísticas de combate. Inconsistências estruturais e registros incompletos foram tratados via imputação numérica por preenchimento com zero.

Em seguida, no Módulo 2 são selecionadas as características que servirão de entrada (X) para os classificadores. Para o *Titanic* utilizaram-se cinco atributos contínuos/discretos (*Pclass, Age, SibSp, Parch e Fare*) combinados à codificação binária de sexo (*Sex*). Para o *Pokémon*, empregaram-se as seis estatísticas numéricas de combate e dois atributos físicos além da codificação *One-Hot* da cor (*Color*). Executa-se a divisão

dos dados em treino e teste sob amostragem estratificada (na proporção de 80% e 20%, com semente pseudoaleatória fixada em 42 para garantir reprodutibilidade dos experimentos) e a formatação do vetor de saída (*target*, y) foi extraída a partir da coluna de sobrevivência *Survived* para o *Titanic* e da tipagem primária *Type1* para o *Pokémon*. Ambas foram codificadas em representação binária *One-Hot* para a rede neural e em rótulos inteiros para o modelo bayesiano. Para evitar a saturação da ativação sigmoidal, todas as variáveis contínuas foram normalizadas linearmente para o intervalo $[0, 1]$ via escalonamento *Min-Max*, com mínimo e máximo calculados apenas no conjunto de treino e aplicado também ao de teste, evitando vazamento de informação.

B. Escolha do Algoritmo e Seleção de (Hiper)Parâmetros

No Módulo 3 ocorre a escolha dos algoritmos, no caso deste trabalho foram selecionados: uma rede neural artificial MLP com *backpropagation* e o classificador probabilístico Naive Bayes Gaussiano. Já no Módulo 4 as variáveis de configuração do processo de aprendizagem foram ajustadas a partir de experimentos exploratórios preliminares. Na MLP, analisou-se a variação da camada oculta (16, 32 e 64 neurônios) e da taxa de aprendizado ($\eta \in \{0{,}01;\ 0{,}05;\ 0{,}1;\ 0{,}15;\ 0{,}5\}$). Concluiu-se que arquiteturas menores (16 neurônios) apresentaram dificuldade em convergir no espaço multiclasse do conjunto *Pokémon*. Por outro lado, a configuração com 64 neurônios exibiu sobreajuste e lentidão de convergência devido à saturação das ativações sigmoides frente ao número reduzido de amostras por classe, resultando em perda de generalização no teste. Em relação às taxas de aprendizado taxas menores ($0{,}01$ e $0{,}05$) tornaram a descida do gradiente excessivamente lenta ao longo das épocas sem ganhos significativos de acurácia, taxas elevadas como $0{,}5$ causaram oscilações na convergência da perda. Os valores intermediários ($0{,}1$ e $0{,}15$) apresentaram comportamento estável. A configuração final fixou a taxa de aprendizado em $\eta = 0{,}1$, 32 neurônios na camada oculta, ativação sigmoide, limite de $10\,000$ épocas e limiar de convergência (paciência) de $0{,}01$, proporcionando equilíbrio entre estabilidade numérica e velocidade de convergência. Os pesos e vieses foram inicializados aleatoriamente a partir de distribuição uniforme no intervalo $[-0{,}5;\ +0{,}5]$. No Naive Bayes definiu-se a constante de suavização ($0{,}15$) adicionada ao desvio padrão para prevenir divisões por zero.

C. Treinamento

No Módulo 5 ocorre a execução do processo em que o algoritmo ajusta os seus parâmetros internos, no caso da MLP tem-se a atualização dos seus pesos W_1, W_2 e dos vieses b_1, b_2 via cálculo dos gradientes locais com o algoritmo *backpropagation* estocástico, já no Naive Bayes ocorre o cálculo direto da média (μ), do desvio padrão (σ) e das frequências a *priori* $P(C_k)$ para cada classe.

D. Avaliação de Métricas

Por fim, no Módulo 6, decorre a avaliação de desempenho do modelo treinado sobre o conjunto de teste inédito. Por meio da extração da matriz de confusão, foram adquiridos os valores de Verdadeiros Positivos (TP), Verdadeiros Negativos (TN), Falsos Positivos (FP) e Falsos Negativos (FN). Com esses valores foram calculadas quatro métricas globais e por classe:

$$\text{Acurácia} = \frac{\sum_{k=1}^{K} TP_k}{N} \tag{11}$$

$$\text{Precisão} = \frac{TP}{TP + FP} \tag{12}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{13}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}} \tag{14}$$

Enfim realizou-se a análise da curva de perda (*Loss*) para verificação de convergência na MLP.

IV. Resultados e Discussão

A seguir, são comentados os resultados obtidos experimentalmente ao treinar e testar os algoritmos MLP e Naive Bayes nas bases de dados descritas na Seção III.

A. Dinâmica de Treinamento e Convergência da MLP

As Figuras 2 e 3 mostram o erro total por época. Em ambas as bases, o erro cai rapidamente no início e depois diminui lentamente, sem oscilações, o que indica que $\eta = 0{,}1$ é estável. Ao final das $10\,000$ épocas, o erro total ainda era de cerca de $71{,}5$ no *Titanic* e de 70 no *Pokémon*, muito acima do limiar de $0{,}01$, de modo que o treinamento foi encerrado pelo limite de épocas e as curvas apenas tendem à estabilização.

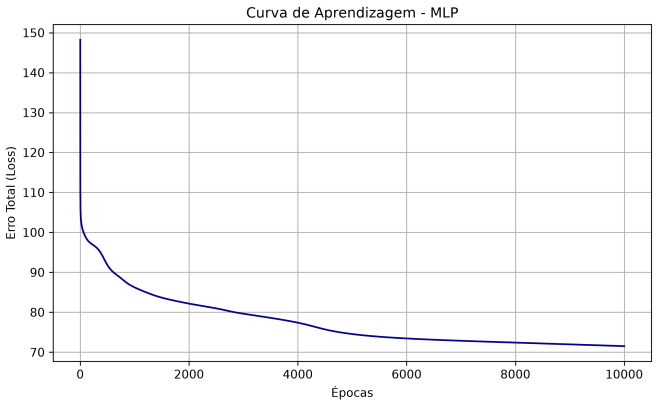


Figura 2: Curva de Aprendizagem - Base de Dados Titanic.

B. Avaliação no Dataset Titanic (Classificação Binária)

A partir da Tabela I e das matrizes de confusão (Figura 4) observa-se que a MLP superou levemente o Naive Bayes na acurácia global ($79{,}89\%$ na MLP e $78{,}77\%$ no Naive Bayes). Essa diferença corresponde a apenas duas amostras do conjunto de teste, de modo que o desempenho dos dois classificadores pode ser considerado equivalente nessa base.

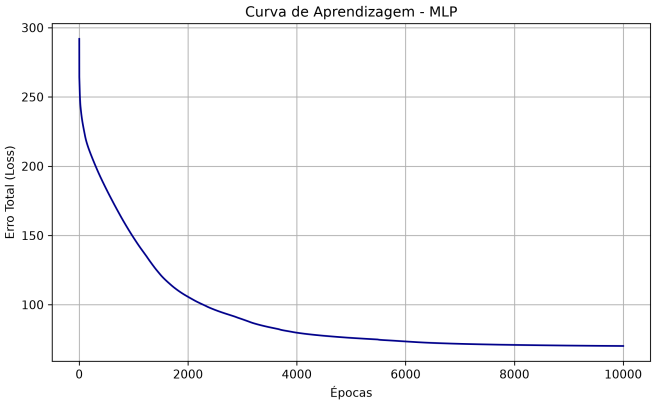


Figura 3: Curva de Aprendizagem - Base de Dados Pokémon.

Ambos ficaram mais de 17 pontos percentuais acima de um classificador que sempre prediz a classe majoritária *Não Sobreviveu*, cuja acurácia seria de $110/179 \approx 61{,}45\%$.

Na classe de *Sobreviveu*, ambos os classificadores acertaram a mesma quantidade de sobreviventes (46 de 69), indicado no índice de *recall* (67% para ambos). A diferença entre os modelos aparece na classe *Não Sobreviveu*, em que a MLP acertou 97 das 110 amostras (*Recall*: 88%) e o Naive Bayes 95 (86%), e na precisão da classe *Sobreviveu* (78% da MLP e 75% do Naive Bayes), dois falsos positivos a mais. Nos dois casos, o desempenho da classe *Sobreviveu* é inferior ao da classe *Não Sobreviveu*, indicando uma tendência a favorecer a classe com mais amostras.

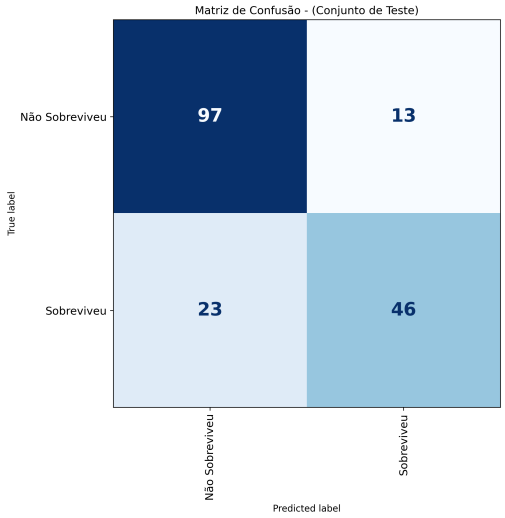
A suposição de independência condicional do Naive Bayes, que desconsidera a dependência entre atributos como classe econômica, tarifa e sexo, poderia explicar o desempenho ligeiramente inferior. No entanto, dada a pequena diferença, os resultados sugerem que, nessa base, a capacidade da MLP de modelar relações mais complexas entre os atributos trouxe pouco ganho.

Tabela I: Comparação de Desempenho no Dataset Titanic

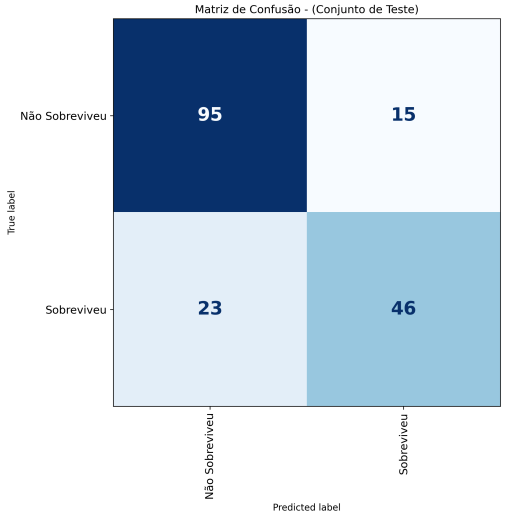
Classe	**MLP**			**Naive Bayes**			**Qtd. Amostras**
	Prec.	**Rec.**	**F1**	**Prec.**	**Rec.**	**F1**	
Não Sobrev.	0,81	0,88	0,84	0,81	0,86	0,83	110
Sobreviveu	0,78	0,67	0,72	0,75	0,67	0,71	69
Acurácia		**0,7989**			**0,7877**		179

C. Avaliação no Dataset Pokémon (Classificação Multiclasse)

Como a classificação dos *Pokémon* é um problema multi-classe, nota-se uma diferença maior entre os algoritmos. A acurácia global da MLP foi de $35{,}17\%$ contra $17{,}24\%$ do Naive Bayes. Ao comparar com o desempenho esperado ao acaso ($1/18 \approx 5{,}56\%$) e com o de um classificador que sempre prediz a classe majoritária (*Water* $\approx 14{,}5\%$), a MLP fica mais de seis vezes acima do acaso e cerca de duas vezes e meia



(a) Algoritmo MLP.



(b) Algoritmo Naive Bayes.

Figura 4: Comparação das matrizes de confusão na base Titanic.

acima da classe majoritária, enquanto o Naive Bayes supera esta última apenas ligeiramente.

Conforme a Tabela II, nota-se que o Naive Bayes teve zero de precisão, *recall* e *F1-Score* em 11 das 18 classes. A matriz de confusão (Figura 6) mostra que o modelo concentrou suas predições em poucas classes: *Fire*, *Fairy* e *Water* receberam cerca de 66% de todas as predições, resultando em um *recall* elevado mas precisão baixa nessas classes, enquanto classes como *Normal*, com 19 amostras não foram acertadas. Esse comportamento pode estar associado à suposição de distribuição gaussiana dos atributos e à estimativa pouco confiável de médias e desvios em classes com poucas amostras.

Os resultados da MLP foram mais distribuídos entre as classes, conforme ilustrado na Figura 5, tendo apenas três com métricas zeradas (*Flying*, *Ghost* e *Ice*, variando de 1

Tabela II: Comparação de Desempenho no Dataset Pokémon

Classe	MLP			Naive Bayes			Qtd. Amostras
	Prec.	Rec.	F1	Prec.	Rec.	F1	
Bug	0,11	0,08	0,09	0,00	0,00	0,00	13
Dark	0,17	0,17	0,17	0,00	0,00	0,00	6
Dragon	0,60	0,60	0,60	0,00	0,00	0,00	5
Electric	0,25	0,43	0,32	0,00	0,00	0,00	7
Fairy	0,14	0,33	0,20	0,14	1,00	0,25	3
Fighting	0,25	0,20	0,22	0,00	0,00	0,00	5
Fire	0,40	0,44	0,42	0,16	1,00	0,28	9
Flying	0,00	0,00	0,00	0,00	0,00	0,00	1
Ghost	0,00	0,00	0,00	0,20	0,40	0,27	5
Grass	0,55	0,46	0,50	0,50	0,08	0,13	13
Ground	0,38	0,50	0,43	0,00	0,00	0,00	6
Ice	0,00	0,00	0,00	0,00	0,00	0,00	5
Normal	0,52	0,58	0,55	0,00	0,00	0,00	19
Poison	0,50	0,33	0,40	0,18	0,33	0,24	6
Psychic	0,25	0,22	0,24	0,00	0,00	0,00	9
Rock	0,33	0,25	0,29	0,00	0,00	0,00	8
Steel	0,50	0,50	0,50	0,20	0,25	0,22	4
Water	0,36	0,43	0,39	0,37	0,33	0,35	21
Acurácia Global	**0,3517**			**0,1724**			**145**

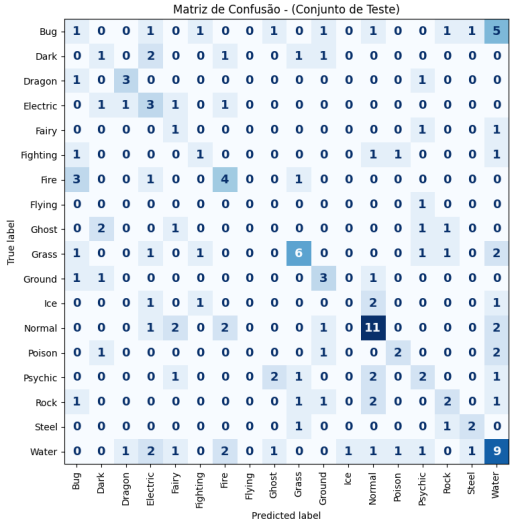


Figura 5: Matriz de Confusão - Base de Dados Pokémon (Algoritmo MLP).

a 5 amostras), e melhores resultados em *Dragon* (*F1-Score* de 0,60), *Normal* (0,55), *Grass* e *Steel* (0,50 cada), embora algumas dessas classes tenham poucas amostras e seus valores sejam sensíveis a poucos acertos. A curva de aprendizagem (Figura 3) mostra que o erro total caiu rapidamente no início e passou a diminuir lentamente, terminando próximo de 70, bem acima do limiar de 0,01, de modo que o treinamento foi encerrado pelo limite de épocas. A acurácia global foi baixa (35,17%), o que era esperado dada a baixa quantidade de amostras por classe e a complexidade do problema, visto que os atributos dos *Pokémon* tem relação apenas parcial com o tipo, além de muitos possuírem um segundo tipo não considerado no teste.

V. Conclusão

Este artigo apresentou um estudo comparativo entre os classificadores Multilayer Perceptron e Naive Bayes Gaussiano, aplicados à classificação binária (*Titanic* [1]) e multiclasse

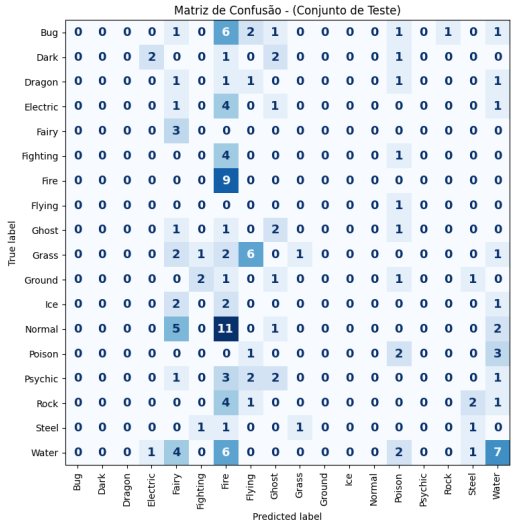


Figura 6: Matriz de Confusão - Base de Dados Pokémon (Algoritmo Naive Bayes).

(*Pokémon* [2]), avaliados por acurácia, precisão, *recall*, *F1-Score*, matrizes de confusão e curva de aprendizagem.

Os dados foram divididos de forma estratificada em treino e teste, normalizados por *Min-Max* com parâmetros obtidos apenas no treino, e os atributos categóricos foram codificados com *one-hot*. A MLP e o Naive Bayes foram implementados do zero, conforme a Seção II, e os hiperparâmetros foram ajustados por tentativa e erro.

No *Titanic*, os modelos tiveram desempenho equivalente: 79,89% para a MLP contra 78,77% para o Naive Bayes, diferença de apenas duas amostras. No *Pokémon*, a MLP alcançou 35,17%, mais de seis vezes o acaso (5,56%), enquanto o Naive Bayes obteve 17,24%, pouco acima da classe majoritária (14,5%), com métricas nulas em 11 das 18 classes e predições concentradas em poucas delas. Os resultados sugerem que a MLP é mais adequada a problemas complexos, ao custo de maior esforço computacional, e que o Naive Bayes é uma alternativa acessível e de baixo custo em problemas mais simples.

Essas conclusões são limitadas pelo uso de apenas duas bases, de uma única partição treino/teste, do ajuste de hiperparâmetros por análise exloratória e do rótulo restrito ao tipo primário no *Pokémon*. Como trabalhos futuros, sugerem-se a validação cruzada, a variação da arquitetura da MLP e o uso de distribuições adequadas a atributos binários no Naive Bayes.

Transparência no Uso de IA Generativa

Durante a escrita do trabalho, foram utilizados os modelos de IA Generativa (Gemini e Claude) para revisão gramatical, aperfeiçoamento e assistência na formatação em LaTex. Na confecção do código os mesmos modelos foram utilizados para tirar dúvidas sobre as bibliotecas e erros do código. Os autores revisaram e alteraram os conteúdos conforme a necessidade e assumem total responsabilidade sobre o conteúdo do artigo.

REFERÊNCIAS

[1] Kaggle, "Titanic - Machine Learning from Disaster", 2026. Disponível em: https://www.kaggle.com/datasets/yasserh/titanic-dataset. Acesso em: 31 out. 2026.

[2] Kaggle, "Complete Pokemon Dataset", 2026. Disponível em: https://www.kaggle.com/datasets/alopez247/pokemon. Acesso em: 31 out. 2026.

[3] T. M. Mitchell, *Machine Learning*, 1st ed. New York, NY, USA: McGraw-Hill, 1997.

[4] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[5] W. McKinney, "Data structures for statistical computing in Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61.

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.